

I. NECESSITE DE LA PROGRAMMATION FONCTIONNELLE

1. NOTION D'EFFETS DE BORD

Exo 1.

1. Donner un pseudo-code possible pour une fonction **mediane(L)** qui renvoie la médiane d'une liste L de nombres passée en paramètres (sans recopier la liste).
2. Le tableau **age** donne son âge pour chaque marin de l'équipage. Le développeur a positionné le capitaine à l'indice 0, afin d'avoir un accès immédiat à l'âge du capitaine. Cette contrainte est-elle compatible avec l'utilisation de la fonction **mediane()** qui permettrait d'obtenir l'âge médian de l'équipage ?

En plus de renvoyer une valeur, en général premier effet recherché dans une fonction, une fonction peut entraîner des modifications au-delà de sa portée comme :

- ✚ modifier des variables globales ;
- ✚ modifier des arguments mutables (i.e. modifiables);
- ✚ afficher des informations à l'écran ;
- ✚ enregistrer des données dans un fichier.

Une fonction est dite à effet de bord si elle modifie un état autre que sa valeur de retour. On peut alors parler de fonction impure.



Souvent, ces effets compliquent la lisibilité du comportement des programmes et/ou nuisent à la réutilisabilité des fonctions et procédures :

Ex 1 : Dans l'exemple suivant, le résultat de l'appel à **f()** dépend du moment où on l'appelle :

```
i = 0
def f():
    global i
    i = i + 1
    return i

f() - f() # (1 - 2) or (2 - 1) ?
```



En Python : -1
En C : 1 ou -1

Rq1. Un effet de bord apparaît souvent lorsqu'une modification d'un programme cohérent (valeurs et états pris conformes aux spécifications) aboutit à des valeurs ou des comportements non prévus, à cause de la non prise en compte de la portée, de l'ensemble de définition de variables, ou du contrat des fonctions.

Rq2. L'expression « Effet de bord » est issue d'une traduction approximative de l'anglais *side-effect*. (Le mot *side* signifiant " côté " et non pas " bord "). Une traduction littérale et correcte aurait donné " effet latéral " ; cependant, en français, on utilise plus fréquemment l'expression synonyme " effet secondaire ", qui peut aussi s'apparenter parfois à un dégât collatéral...

La programmation impérative, en particulier lors de l'emploi de variables globales, provoque des effets de bord dans le fonctionnement de ses programmes mais il est impossible pour un programme sans effets de bord de faire quelque chose d'utile (car il faut au moins afficher le résultat). Il est cependant possible de limiter les effets de bord à une petite partie du programme. C'est donc ce que tente de faire la programmation fonctionnelle

2. PRINCIPES DE LA PROGRAMMATION FONCTIONNELLE

La programmation fonctionnelle est un paradigme de programmation au même titre que la programmation impérative ou la programmation orientée objet.

La programmation fonctionnelle s'appuie sur deux principes :

- ✚ le résultat de l'évaluation d'une fonction dite « pure » ne dépend que de la valeur de ses paramètres, pas de facteurs externes ;
- ✚ les fonctions sont les briques de base pour composer les expressions ; elles peuvent apparaître comme paramètres, retours ou données d'un programme.

Rq3. Certains langages de programmation (les langages fonctionnels) ont été conçus pour imposer au programmeur le paradigme fonctionnel. C'est le cas en particulier de Haskell, ML, ou Scheme....

Rq4. Il est possible d'utiliser la programmation fonctionnelle avec un langage qui permet les effets de bord, mais il sera souvent difficile pour le compilateur de *prouver* l'absence d'effets de bord. Il est beaucoup plus facile pour le programmeur de se convaincre de l'absence d'effets de bord et cela suffit souvent pour obtenir un programme plus compréhensible.
C'est en tout cas ce que l'on s'efforcera de faire en terminale et en *python*.

II. NON AUX EFFETS DE BORD

1. FONCTIONS PURES

Exo 2. Pure ou impure ?

Préciser dans chaque cas si la fonction est pure (elle respecte le paradigme fonctionnel) ou impure ? Justifier

<pre>i = 5 def fct(): if i > 5: return True else : return False fct()</pre>	<pre>def fct(i): if i > 5: return True else : return False fct(5)</pre>	<pre>l = [4,7,3] def ajout(i): l.append(i)</pre>	<pre>def ajout(i,l): tab = l + [i] return tab</pre>



Revient à considérer les fonctions comme leurs homologues mathématiques : des applications.

Rq5. Comme on le voit sur les deux dernières fonctions, le paradigme fonctionnel va amener le programmeur non pas à modifier une valeur existante (ce que fait la méthode **append** sur la liste **l**), mais plutôt à créer une nouvelle grandeur à partir de la grandeur existante (ce que fait avec l'opérateur **+** et l'affectation de la nouvelle liste **tab**) : une grandeur existante n'est jamais modifiée. Il n'y a donc aucun risque d'effet de bord.)

Rq6. Il n'est pas toujours simple d'écrire des fonctions pures, il faut parfois faire des concessions à l'encontre du paradigme.

2. STRUCTURES DE DONNEES MUABLES OU IMMUABLES

Une structure de données immuables (en anglais *immutable*) est une structure de données, a priori quelconque (tableau, ensemble, dictionnaire, etc.), que l'on ne peut plus modifier une fois construite. On peut en revanche la lire et en déduire éventuellement d'autres structures.

A contrario, un objet muable (en anglais *mutable*) est ainsi un objet qui peut être modifié, dont on peut changer les propriétés une fois qu'il a été défini.



Une erreur courante est de confondre modification et réassignation...

La différence est facile à comprendre avec les listes. Les listes sont des objets muables : une fois la liste instanciée, il est par exemple possible d'y insérer de nouveaux éléments.

```
>>> values = [0, 1, 2]
>>> values.append(3)
>>> values
[0, 1, 2, 3]
```

Le fonctionnement des variables en Python fait qu'il est possible d'avoir plusieurs noms (étiquettes) sur une même valeur. Le principe de mutabilité s'observe alors très bien.

```
>>> values = [0, 1, 2]
>>> othervalues = values
>>> values.append(3)
>>> values
[0, 1, 2, 3]
>>> othervalues
[0, 1, 2, 3]
```

Sans avoir explicitement touché à **othervalues**, sa valeur a changé. En effet, **values** et **othervalues** référencent un même objet. Un même objet muable.

En revanche, la réassignation fait correspondre le nom de la variable à un nouvel objet, il ne s'agit pas d'une modification de la valeur initiale.

```
>>> values = [0, 1, 2]
>>> othervalues = values
>>> values = [0, 1, 2, 3] # réassignation de values
>>> values
[0, 1, 2, 3]
>>> othervalues
[0, 1, 2]
```

En Python, les objets de type **bool**, **int**, **str**, **bytes**, **tuple**, **range** et **frozenset** sont immuables. Tous les autres types, les listes, les dictionnaires, ou les instances de vos propres classes sont des objets muables.

Ex 2 : Des entiers :

```
>>> i=0
>>> i
0
>>> id(i)
2078794544
>>> i=i+1 #crée une autre instance de l'objet i
>>> i
1
>>> id(i)
2078794560
```

Comme vu précédemment, les objets mutables sont à prendre avec précautions, car leur valeur peut changer sans que l'avoir explicitement demandé. Cela peut arriver lorsqu'une valeur mutable est passée en argument à une fonction.

```
def append_42(values):
...   values.append(42)
...   return values
...
>>> v = [1, 2, 3, 4]
>>> append_42(v)
[1, 2, 3, 4, 42]
>>> v
[1, 2, 3, 4, 42]
```

Cela ne pourra jamais arriver avec un tuple par exemple, qui est immuable et ne possède aucune méthode pour être altéré.

```
def append_42(values):
...   return values + (42)
...
>>> v = (1, 2, 3, 4)
>>> append_42(v)
(1, 2, 3, 4, 42)
>>> v
(1, 2, 3, 4)
```

Le paradigme fonctionnel encourage donc pour éviter les effets de bords à manipuler et transformer des objets immuables

3. UN EXEMPLE



On cherche à programmer une structure de données pour réaliser des ensembles. On fournit en particulier une opération pour construire un ensemble vide, pour ajouter, pour supprimer un élément et une opération pour tester si un élément appartient à un ensemble. On ne s'intéresse pas ici à l'implémentation mais uniquement à l'interface utilisateur de ces opérations selon le paradigme de programmation choisi.

a) Paradigme impératif

```
s = créer_ensemble()
ajouter(s,34)
ajouter(s,45)
```



l'opération **ajouter()** modifie l'ensemble **s** qu'elle a reçu en argument, pour lui en ajouter un nouvel élément.

b) Paradigme objet

```
s = Ensemble()   #instanciation (constructeur)
s.ajouter(34)     #méthode d'instance
s.ajouter(45)

# identique à Ensemble.ajouter(s,34)
```



Phénomène assez semblable à la syntaxe près.

c) Paradigme fonctionnel

```
s0 = créer_ensemble()  #instanciation
                           (constructeur)
s1=ajouter(s0,34)
s2=ajouter(s1,34)
```



A chaque appel, un nouvel ensemble est créé.

Rq7. De manière générale, la programmation fonctionnelle se caractérise par l'idée que, comme en maths, quand on applique deux fois la même fonction aux mêmes arguments alors on obtient le même résultat.

Rq8. Finalement l'intérêt de la « pureté » se manifeste par :

- ✚ La **Portabilité** de la fonction : indépendance du moment et du lieu de l'appel.
- ✚ La Facilitation des **tests** : pas de nécessité de préparer un contexte particulier à chaque fois.
- ✚ La **Parallélisation** possible du code : des appels indépendants peuvent être faits sur des machines différentes.

III. FONCTIONS COMME ARGUMENTS OU COMME RETOURS

1. FONCTION PASSEE EN ARGUMENT

Une fonction peut être paramétrée par une autre fonction.

Rq9. C'est alors un mécanisme qui peut s'apparenter à une composition mathématique de fonctions.

a) Exemple du tri générique

Ex 3 : un algorithme de tri paramétré par un **ordre** de tri

On considère l'algorithme de tri des éléments d'un tableau `t` : [fc en argument - ex du tri.py](#)

```
def estTrie(t):
    for i in range(len(t)-1):
        if t[i] > t[i+1]:
            return False
    return True

def insere(t, i, v):
    """insère v dans t[0..i[ supposé trié"""
    j = i
    while j > 0 and t[j - 1] > v:
        t[j] = t[j - 1]
        j = j - 1
    t[j] = v

def tri_insertion(t):
    """trie le tableau t dans l'ordre croissant"""
    for i in range(1, len(t)):
        insere(t, i, t[i])
        # invariant : t[0..i+1[ est trié
        assert estTrie(t[:i+1])
    #postcondition
    assert estTrie(t)

T=[72, 39, 28, 59, 17, 54]
tri_insertion(T)
print(T)
```



La fonction **tri_insertion(t)** trie le tableau `t`, « en place », dans l'ordre croissant. Cette fonction trie les éléments du tableau à l'aide de l'opérateur `>` de *Python*.

Elle possède un certain degré de généricité dans la mesure où elle permet de trier aussi bien des entiers que des flottants ou des strings (l'opérateur `>` de *Python* applique un ordre sur les nombres ou sur les strings (qui est alors l'ordre alphabétique) :

```
>>> "abc"<"adc" # < se lit comme « précède dans l'ordre alphabétique
True
>>> "abc"<"ab"
False
```



Elle fonctionne également sur les n-uplets par application de l'ordre lexicographique :

```
>>> (3,4,6) < (7,4,2)
True
>>> (3,4,6) < (3,4,2)
False
```



On peut cependant vouloir adapter la relation d'ordre utilisée pour le tri au type de données à trier :

Ex 4 : Si l'on dispose par exemple d'un tableau contenant des triplets (nom, date de naissance, date de décès),

```
histoire=[ ('Jobs', 1955, 2011),
            ('Dijkstra', 1930, 2002),
            ('Turing', 1912, 1954),
            ('Hopper', 1906, 1992,)]
```



on n'obtiendra pas un tri sur les dates de naissances en utilisant l'opérateur > de Python. Il s'agit donc de redéfinir **la relation d'ordre** utilisée pour le tri.

Exo 3. Sur la base de la fonction `tri_insertion`, ([fc en argument - ex du tri TBC.py](#))

1. créer une fonction `tri_ins_naissance_cr(h)` qui trie le tableau `h` de triplets (nom, date de naissance, date de décès) suivant les dates de naissances croissantes.
2. créer une fonction `tri_ins_deces_decr(h)` qui trie le tableau `h` de triplets (nom, date de naissance, date de décès) suivant les dates de décès décroissantes.

La duplication de code constatée ici peut être évitée en python en passant en paramètre à la fonction de tri non seulement le tableau à trier mais aussi **la fonction de comparaison** entre deux éléments du tableau utilisée comme critère de tri.

3.
 - a. définir par exemple la fonction `ordre_naissance_cr(individu_A, individu_B)` où `individu_A` et `individu_B` sont des triplets (nom, date de naissance, date de décès) qui retourne vrai si `individu_A` est né avant `individu_B`, faux sinon
 - b. créer une fonction `tri_ins_generique(ordonne,t)` qui trie le tableau `t` dans l'ordre défini par la relation d'ordre définie par la fonction booléenne `ordonne()`
Modifier également la fonction `insere()` et tous les appels aux fonctions modifiées.
 - c. Utiliser la fonction `tri_ins_generique(ordonne,t)` pour trier suivant les dates de décès décroissantes, puis dans l'ordre alphabétiques des noms.

b) Utilisation des fonctions anonymes (lambda)



Afin d'éviter de définir une fonction par critère de tri, on peut utiliser la notion de fonction anonyme, fonction sans nom définie par une construction lambda. Ainsi le code

```
def ordre_naissance_cr(individu_A, individu_B) :
    """retourne vrai si individu_A est né avant individu_B, faux sinon"""
    return (individu_A[1]<individu_B[1])

tri_ins_generique(ordre_naissance_cr,
                  histoire)
```

peut être remplacé par un code qui va définir le critère de tri, à la volée, lors du passage de paramètres :

```
tri_ins_generique(lambda individu_A,individu_B:individu_A[1]<individu_B[1] ,
                 histoire)
```

ou plus sobrement :

```
tri_ins_generique(lambda x,y : x[1]<y[1] ,
                 histoire)
```

Exo 4. Tester le code précédent pour en faire un tri décroissant.

c) Exemple du graphe d'une fonction

Ex 5 : [fc en argument - graphe d'une fonction.py](#)

Examiner et tester le code proposé. Préciser quelle partie du code trouve sa place dans ce paragraphe du cours.

2. UNE FONCTION COMME RETOUR DE FONCTION

Ex 6 : de la fonction dérivée d'une fonction :

- a. Renseigner la docstring de la fonction `derive()`, tester le code.

```
def derive(f):
    """ docstring à renseigner """
    h=1e-7
    return lambda x:(f(x+h)-f(x))/h

from math import sin, cos, pi, exp

d=derive(sin)
print("nb dérivé de sin en {} = {}".format(0.,d(0.)))
print("nb dérivé de sin en {} = {}".format(pi/3,d(pi/3)))

d=derive(lambda x:exp(-x))
print("nb dérivé de x:exp(-x) en {} = {}".format(0.,d(0.)))
print("nb dérivé de x:exp(-x) en {} = {}".format(1.,d(1)))
```

- b. Les retours de la console sont-ils conformes aux résultats attendus ?

IV. EXOS :

Exo 5. Ecrire une fonction **compose(f,g)** qui prend en arguments deux fonctions f et g de la variable réelle à valeur réelle et qui renvoie la fonction $f \circ g$ (c'est-à-dire la fonction $x \mapsto f(g(x))$)

Exo 6. Ecrire une fonction **temps_d_execution(f,x)** qui prend en arguments une fonction f et une valeur x et qui renvoie l'image $f(x)$ et le temps mis pour effectuer ce calcul. Pour mesurer ce temps, on pourra utiliser la fonction **timeit.timeit()** ou la fonction **perf_counter()** du module **time**. Mesurer le temps pris par la fonction **boucle_inutile()** sur l'argument 10 000 000

```
def boucle_inutile (n):
    k=0
    for i in range(n):
        k=k+1
```

Exo 7. Le paradigme de la programmation événementielle consiste à associer du code à des événements afin que ce code soit exécuté lorsque l'événement survient. (c'est ce qui a été fait en première en python avec TKInter ou en programmation web avec JavaScript). C'est une application naturelle de la programmation fonctionnelle : le code associé aux événements est une fonction.

- Compléter le code Python `tkinter_compteur_de_clics_TBC.py` pour qu'à chaque clic de bouton, le nombre affiché sur celui-ci soit incrémenté.
- Modifier le code afin de visualiser une rangée de 10 boutons, chacun affichant son compteur de clics indépendant de ses camarades boutons.



Essayer d'éviter les variables globales.

